

Software Lifecycle Management



Fachbereich Informatik und Automation (IAT)

Dr. rer. nat. Nils Beckmann
nils.beckmann@th-owl.de

Anforderungen

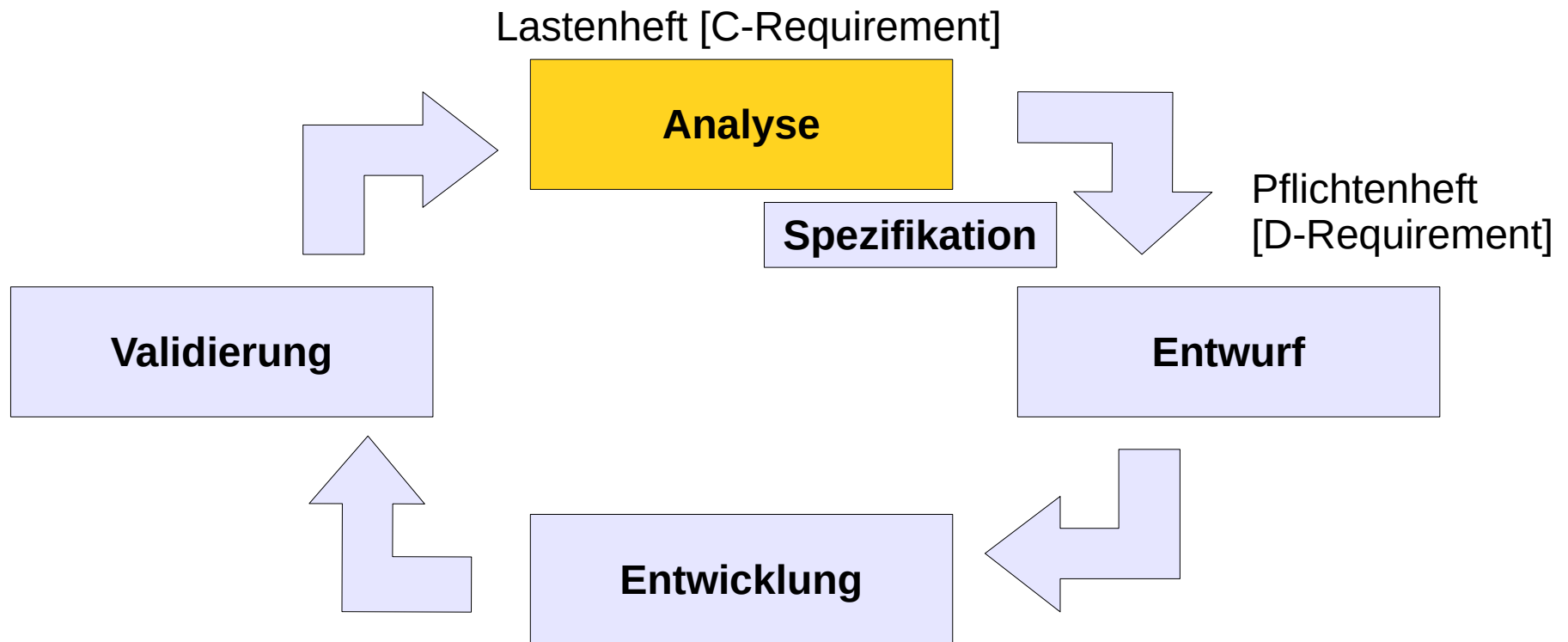
Agenda

- Requirements Management (REQ)
 - Von der Idee zur Anforderung
 - Die Softwareanforderungsspezifikation (SAS)
 - Lastenheft und Pflichtenheft
- Anforderungen
 - Arten: Benutzer-/System-, (Nicht-)Funktional
 - Kriterien für gute Anforderungen
 - Formulierungen und Beispiele
 - Metriken für Nicht-Funktionale Anforderungen

Erinnerung: "Software-Prozess"

- "Ein **Softwareprozess** ist als eine Menge von Tätigkeiten definiert, die zur Produktion eines **Softwareprodukts** führen."

Erinnerung: Softwareprozessphasen



Das Requirements Management ist verantwortlich für die Anforderungen (Requirements) an die Software.

Requirements Management (REQ)



Analyse: Von der Idee zur Anforderung



Ideen,
Brainstorming

- Marktanforderungen
- Ideensammlung
- Visionen entwickeln
- Kundengespräche
- Mitbewerber analysieren
- Innovationen
- Recherchen
- Interviews
- Fragebögen
- Beobachtung
- Berichte einholen
- Inventur



Machbarkeitsanalyse

- Detaillierung der Ideen
- Ideen bewerten und priorisieren
- Anforderungen formulieren
- Reviews und Feedback
- Freigabe durch Auftraggeber und Auftragnehmer

- Technische Machbarkeit
- Finanzieller und zeitlicher Rahmen
- Kompetenzprüfung
- Risikoprüfungen (SWOT, etc.)
- Verfügbarkeit von Ressource



Spezifizierung
der
Anforderungen

Softwareanforderungsspezifikation

- Die **SAS** ist das wichtigste Werkzeug der klassischen Softwareentwicklung. Sie enthält vollumfänglich die Beschreibung zur Erstellung der Software. Darauf basierend kann die Software erstellt werden.
- Beispielstrukturen für SAS:
 - IEEE 830 - Recommended Practice for Software Requirements Specifications 1998
 - ANSI/IEEE Std 29148-2011
 - "Lehrbuch der Software-Technik", Balzert, S. 59 und S. 105
 - Ilias-Lehrbereich "Vorlagen"

Softwareanforderungsspezifikation

- **Softwareanforderungsspezifikation (SAS)**
= **Lastenheft + Pflichtenheft**
- **Lastenheft**
= Customer-Requirement (C-Requirement)
= "Grobentwurf", "Produktentwurf"
 - WAS?
- **Pflichtenheft**
= Developer-Requirement (D-Requirement)
= "Feinentwurf", "Technischer Entwurf",
"Datenverarbeitungsentwurf (DV-Entwurf)"
 - WIE? WOMIT?

Lastenheft

- **Was** soll entwickelt werden?
- „Zusammenfassung aller fachlichen Basisanforderungen an das Softwareprodukt aus Sicht des Auftraggebers.“
 - ... sollte als Grundlage für eine offene Ausschreibung dienen können (für mehrere Bewerber)
 - ... kann als Unterstützung vom Auftraggeber für den Auftragnehmer erstellt werden
- Erstellt von z.B. Kunde, Nicht-Fachmann, Auftraggeber, etc. zusammen mit z.B. Vertriebler, Requirements Engineer, Auftragnehmer, etc.

Pflichtenheft

- **Wie** und **Womit** soll entwickelt werden?
- „Zusammenfassung aller fachlichen Anforderungen an das Softwareprodukt aus Sicht des Auftraggebers inklusive Realisierungsvorgaben aus Sicht des Auftragnehmers.“
 - ... stellt eine deutliche Verfeinerung des Lastenhefts dar
 - ... sollte als Grundlage für einen Vertrag abgefasst sein
 - ... dient zur Abnahme des fertigen Softwareprodukts
 - ... sollte vor Beginn der Implementierung vom Auftraggeber abgenommen werden
 - ... sollte keine wichtigen Entwurfs- oder Implementierungsentscheidungen vorwegnehmen
- Erstellt von z.B. Auftragnehmer, Requirements Engineer, Software-Architekten, etc. basierend auf dem Lastenheft.

Arten von Anforderungen

- Anforderungen werden in **Funktionale** und **Nicht-Funktionale** Anforderungen unterteilt.
- Weiterhin werden Anforderungen in **Benutzeranforderungen** und **Systemanforderungen** unterteilt.

Anforderungen

Benutzer- / Systemanforderungen

- **Benutzeranforderungen** sollten technische Fachbegriffe vermeiden und in natürlicher Sprache geschrieben sein
 - Für Benutzer/Anwender, Manager, etc.
- **Systemanforderungen** beschreiben Anforderungen an die Softwareanwendung in technischer Sprache und mit Fachbegriffen
 - Für Entwickler/Programmierer, Software-/System-Architekten, etc.

Beispiele für Anforderungen

- Benutzeranforderung:
 - Beim Start des Programms soll das Firmenlogo erscheinen.
 - Jede Funktion der Software soll in einem eigenen Fenster erscheinen.
 - In der Titelleiste befindet sich ein Button zu einer Hilfeseite zur Software.
- Systemanforderung:
 - Die Kommunikation der Netzwerkschnittstelle erfolgt über TCP-Port 790.
 - Die über das GUI-Backend erzeugten Daten werden im Pfad "C:\Daten\Log.txt" abgelegt.

Anforderungen

Funktional vs. Nicht-Funktional

- Funktionale Anforderungen
 - Aussagen zu Diensten, die das System leisten soll
 - Reaktion des Systems auf bestimmte Eingaben
 - Verhaltens des Systems in bestimmten Situationen
- Nicht-Funktionale Anforderungen
 - Randbedingungen an das System (Zeit, bestimmte Standards, Speicher und Geschwindigkeit, usw.)
 - Beziehen sich meist auf das ganze System und nicht auf spezielle Aufgaben

Kriterien für gute Anforderungen

- Eine **Gute SAS** ist durch ihre Anforderungen:
 - 1) Korrekt
 - 2) Eindeutig
 - 3) Vollständig
 - 4) Konsistent
 - 5) Bewertet: Bedeutung, Stabilität, Risiko, Reihenfolge
 - 6) Testbar
 - 7) Modifizierbar
 - 8) Nachvollziehbar
 - 9) Verfolgbar

Kriterien für gute Anforderungen

- 1) **Korrekt** = Jede Anforderung sollte in der Software umgesetzt/erfüllt sein
- 2) **Eindeutig** = Jede Anforderung erlaubt genau eine Interpretation
- 3) **Vollständig** = Alle zu erfüllenden Anforderungen sind enthalten, inklusive Hinweisen, Referenzen, wichtigen Umgebungs-/Nebenbedingungen, Details, usw.
- 4) **Konsistent** = Anforderungen widersprechen sich nicht

Kriterien für gute Anforderungen

- **5) Bewertet:**

- Bedeutung = z.B. "Must have", "Should have", "Nice to have", z.B. auch Bewertung nach Relevanz mit Zahl [1;99]
- Stabilität = z.B. "Review OK", "Review Required", "Re-Review", m.a.W.: Wie sicher kann die Anforderung sicher erfüllt werden
- Risiko = z.B. "Umsetzungsdauer nicht/schwierig vorhersagbar", "Abhängigkeiten zu anderen Projekten/Normen", "Hoher Grad der Unwissens zum Umfeld dieser Anforderung", etc.
- Reihenfolge = z.B. Bewertung nach Zahl [1;99] priorisiert die Reihenfolge der Umsetzung

Kriterien für gute Anforderungen

- 6) **Testbar** = Man kann klar einen Test dafür formulieren und so zu einem Ergebnis "erfüllt" oder "nicht erfüllt" kommen (z.B. quantitative Angabe) – nicht testbar sind schwammige Formulierungen wie "einfache Bedienung", "gut funktioniert", "sollte schnell sein", etc.
- 7) **Modifizierbar** = Eine Anforderung kann einzeln und unabhängig von anderen Anforderungen verändert werden
- 8) **Nachvollziehbar** = Es ist erkennbar, aus welchem Grund diese Anforderung aufgenommen wurde (ggf. Wann, von Wem, z.B. Welcher Workshop zugrunde liegt, mit welchem Ziel, etc.)
- 9) **Verfolgbar** = Während des Entwicklungsprozesses kann der Stand der Umsetzung der Anforderung abgefragt werden

Probleme bei Anforderungen

- Mangel an **Genauigkeit** (Details fehlen, Tiefe fehlt, nicht alle ähnlichen Bedeutungen ausgeschlossen, etc.)
- **Relevante Informationen** sind ausgelassen (es fehlen Funktionen, Hinweise auf Umweltbedingungen, Schnittstellen, notwendige Voraussetzungen, etc.)
- **Widersprüche** in den Anforderungen (Unterscheidung und Differenzierung unklar, etc.)
- **Unklare Zuordnungen** und fehlerhafte Struktur (Überschneidungen und unklare Differenzierung von funktionalen sowie nicht-funktionalen Anforderungen, Systemzielen, falscher Kontext, etc.)
- **Redundanz** und Verschmelzung (Mehrfachnennung von Informationen, Ausdifferenzierung schwierig, Kerninformationen schwierig zu erkennen, Informationsüberlastung, etc.)

⇒ Verwirrung, offene Fragen,
Umsetzung nicht möglich, etc.

Formulieren von Anforderungen

"Easy Approach to Requirements Syntax (EARS)"

- Eine Methode zum Formulieren von Anforderungen
- Gibt Satzbausteine vor, die mit individuellen Begriffen aufgefüllt werden müssen
- Durch eine einheitliche Satzstruktur können Unklarheiten, Missverständnisse, Fehler reduziert werden

Easy Approach to Requirements Syntax

- Satzmuster "Universell":
 - "Das <System> muss <Systemreaktion/-eigenschaft>."
 - "Der Geschirrspüler muss eine Tasten zum Einschalten haben."
- Satzmuster "Ereignisgesteuert":
 - "Wenn <Optionale Vorbedingung> <Ereignis>, dann muss <Systemname> <Systemreaktion>."
 - "Wenn der Geschirrspüler eingeschaltet ist und geschlossen wird, soll der Geschirrspülvorgang beginnen."

Easy Approach to Requirements Syntax

- Satzmuster "Zustandsgesteuert":
 - "Solange in <Zustand>, muss <Systemname> <Systemreaktion>."
 - "Solange der Geschirrspüler eingeschaltet ist, blinkt ein LED-Licht grün auf."
- Satzmuster "Unerwünschtes Verhalten":
 - "Falls <Optionale Vorbedingung> <Ereignis>, dann muss <Systemname> <Systemreaktion>."
 - "Falls der Geschirrspüler läuft und geöffnet wird, dann soll sofort die Wasserzufuhr stoppen."

Easy Approach to Requirements Syntax

- Satzmuster "Optional":
 - "Falls <Eigenschaft><Optionale Vorbedingung>, dann muss <Systemname> <Systemreaktion>."
 - "Falls der Geschirrspüler eine Digitalanzeige hat und läuft, soll die Restzeit angezeigt werden."

Uneindeutige Funktionale Anforderungen

- **Unvollständig:**
 - "Der Eingang von Produkten wird überwacht."
 - Wer oder was überwacht das? Was soll geschehen, wenn ein Produkt eingeht und überwacht wird?
- **Implizite Annahmen:**
 - "Die Produktdaten werden in das Eingabeformular eingetragen."
 - Es gibt bestimmte Produktdaten – welche?
Es gibt ein Eingabeformular – wie sieht das aus oder wo ist das abgelegt oder beschrieben?

Uneindeutige Funktionale Anforderungen

- **Implizite Universalisierung:**
 - "Das System sichert alle Daten extern ab, sobald der Benutzer dies wünscht."
 - Welcher Benutzer (Wareneingangsmitarbeiter, Administrator, ...)? Welche Daten (alle Produkte, Login-Daten der Mitarbeiter, Log-Dateien, ...)? Wo ist extern (externe Festplatte, Backup-Laufwerk, Cloud, ...)?
- **Fehlende Spezifizierungen:**
 - "Das System wird bei Störung sofort gestoppt."
 - Was ist eine Störung? Wie sieht diese aus? Was soll beim Stoppen geschehen (schwarzer Bildschirm, System schließen, Computer herunterfahren, ...)?

Detalliertheit vs. Effizienz

- Anforderung: "Die Software zeigt während der Anwendung der Software die aktuelle Uhrzeit in der Statuszeile an."
- **Zu kurz?** "Uhrzeitanzeige" *oder* "Die Software zeigt die Uhrzeit."
- **Zu lang?** "Die Software zeigt die aktuelle Uhrzeit im Format HH:MM rechts unten in der Statuszeile an in der Schriftgröße 12, der Schriftart "Verdana" ohne die Attribute "Fett", "Kursiv", "Durchgestrichen", "Schattiert", "Höher gestellt", "Niedriger gestellt". Die Schriftfarbe ist schwarz und die Anzeige darf beim Wechseln der Minute nicht aufblinken. Die Anzeige erfolgt solange die Software läuft, nicht länger und nicht kürzer. Wenn der Mauszeiger über die Uhrzeitanzeige bewegt wird, darf sich an der Uhrzeitanzeige nichts ändern. Auch wenn innerhalb der Software Fenster geöffnet werden, soll die Uhrzeit trotzdem weiter angezeigt bleiben. Die Zeitzone der Uhrzeit soll angepasst sein an den Standort, an dem die Software gerade genutzt wird."

Detalliertheit vs. Effizienz

- Fragestellung:
 - Ist immer auch Bedeutung, Stabilität, Priorität, Risiko bewertbar – oder notwendig zu bewerten?
 - Ist wirklich jedes Detail notwendig zu definieren?
 - Sind die Kriterien für eine gute Anforderung erfüllt?
 - Sollten bestimmte Dinge vielleicht besser erst später definiert werden, weil noch Informationen fehlen?
 - Vielleicht erst im Pflichtenheft, nach Rücksprache mit Kunden, usw.

Das Optimum liegt meist irgendwo in der Mitte.
⇒ "Goldener Mittelweg"

Metriken für Nicht-Funktionale Anforderungen

- Eine (Software-) **Metrik** ermöglicht die Umwandlung einer **Systemeigenschaft** mittels einer **Maßeinheit quantitativ** auszudrücken und damit zu bewerten – und somit messbar und testbar zu machen.

Metriken für Nicht-Funktionale Anforderungen

- **Geschwindigkeit:** Ausgeführte Transaktionen pro Sekunde, Reaktionszeit auf Benutzereingabe oder Ereignis, Bildschirmauffrischungszeit
- **Größe:** Speicherbedarf in Kilobyte, Anzahl der Speicherbausteine oder Dateien
- **Benutzerfreundlichkeit:** Schulungsdauer, Anzahl der Hilfeseiten
- **Zuverlässigkeit:** Durchschnittliche Zeit bis Fehlfunktion, Wahrscheinlichkeit der Nicht-Verfügbarkeit, Quote für das Auftreten von Fehlern/Meldungen/Warnungen, Verfügbarkeit des Systems (leicht startbar, erreichbar, etc.)
- **Stabilität:** Zeit bis zum Neustart nach Fehlfunktion, Anteil Ereignisse führen zu Fehlfunktion, Wahrscheinlichkeit für Datenzerstörung bei Fehlfunktion
- **Portierbarkeit:** Anteil der plattformabhängigen Anweisungen, Anzahl Zielsysteme
- USW.

Produktqualitätsmatrix

Produktqualität	Kommentar	Hoch	Normal	Nicht relevant
Funktionalität	Alle funktionalen Anforderungen ohne Abstriche notwendig	X		
Leistungseffizienz	Keine besonderen Anforderungen an die Performance		X	
Kompatibilität	Muss nicht mit anderen Systemen zusammenarbeiten			X
Benutzbarkeit	Keine Besonderheiten, intuitiv bedienbar		X	
Zuverlässigkeit	Wichtig, sonst keine Akzeptanz im Team	X		
Sicherheit	Wird nur im internen Netzwerk verwendet, kein Passwort o.Ä. notwendig			X
Änderbarkeit	Sehr wichtig, Software wird ständig weiterentwickelt	X		
Übertragbarkeit	Muss nicht auf andere Systeme übertragen werden			X

*IEC/ISO 25010: Systems and software engineering
Systems and software Quality Requirements and Evaluation (SquaRE)
System and software quality models 2010)*

Untergliederung nicht-funktionaler Qualitätsanforderungen

- "FURPS":
 - F**unctionality (Funktionalität)
 - U**sability (Benutzbarkeit; Erlernbarkeit, Verständlichkeit)
 - R**eliability (Zuverlässigkeit)
 - P**erformance (Effizienz; Leistung, Antwortzeiten, Ressourcenbedarf)
 - S**upportability (Wartbarkeit, Änderbarkeit, Prüfbarkeit)
- Aussehen und Handhabung (Look and Feel)
- Betrieb und Umgebungsbedingungen
- Portierbarkeit, Übertragbarkeit (Anpassbarkeit, Installierbarkeit, Austauschbarkeit)
- Sicherheitsanforderungen (Vertraulichkeit, Datenintegrität)
- Korrektheit (Ergebnisse fehlerfrei)
- Flexibilität (Unterstützung von Standards)
- Skalierbarkeit (Bewältigung einer quantitativen Änderung des Aufgabenumfangs)
- usw.

Zusammenfassung

- Die **Softwareanforderungsspezifikation** (SAS) ist das Kernwerkzeug des Requirements Management (REQ) und beinhaltet das **Lastenheft** und **Pflichtenheft** und enthält damit alle Informationen, die notwendig sind um eine bestimmte Softwareanwendung zu entwickeln.
- In der SAS sind diverse **Anforderungen** enthalten, die die Softwareanwendung mehr oder weniger detailliert beschreiben.
- Die **Anforderung** ist das zentrale Werkzeug des REQ: Es gibt verschiedene **Arten** und **Qualitäten** von Anforderungen.
- Werkzeuge dafür sind Kriterien für gute Anforderungen, „Easy Approach to Requirements Syntax“ (EARS), Metriken für nicht-funktionale Anforderungen, Produktqualitätsmatrix, usw.

Fragen?